Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hafta 3 sınıf/kapsülleme/kurucu–yıkıcı/`this`, Hafta 4 kopya kurucu/RAII/`std::unique_ptr`/`std::vector` ve Hafta 5 "tam bir değer tipi" üstüne kuruludur. Burada tekil bir tipten birden çok *ilişkili* tipi (`Materyal` ailesi) organize etmeye geçilir.

» Kalıtım Nedir? is-a İlişkisi

Temel Fikir ve `public` Kalıtım

Ortak veri/davranışı bir **taban sınıf**ta toplar, farklı olanı **türetilmiş sınıf**larda özelleştirirsiniz (DRY). Ölçüt **is-a**: `BasiliKitap` *bir* `Materyal`'dir. `class BasiliKitap : public Materyal` — türetilmiş, tabanın `public` arayüzünü ve `protected` üyelerini devralır.

» Erişim Düzeyleri

Belirleyici	Sınıfın kendisi	Başka kim erişir
`private`	evet	yalnız kendisi
`protected`	evet	+ türeyenler
`public`	evet	+ herkes (dışarı)

`protected` Ne İşe Yarar

Sınıfa *ve ondan türeyenlere* açık, dışarıya kapalı. `Materyal`'in alanları `protected`'tır ki türetilmişin `yazdir()`'i doğrudan erişsin; `private` olsaydı erişemez, erişimci (`baslik()`) gerekirdi.

» Kurucu Zinciri ve Sıra

```
BasiliKitap(std::string b, Yazar y, int yil, int s)
  : Materyal(std::move(b), std::move(y), yil), // ONCE taban
    sayfaSayisi_(s) { }
```

Kim Önce, Kim Sonra

Kurulumda **taban önce, türetilmiş sonra**; yıkımda **tersten** (bina: önce temel, en son yıkılan temel). Taban kurucusu yazılmazsa tabanın *varsayılan* kurucusu aranır; yoksa **derleme hatası**.

» Nesne Dilimlemesi Tuzağı

Ortak Hata / Tuzak

`Materyal m = bk;` (bk bir `BasiliKitap`) — taban kopya kurucusu *yalnız taban alt-nesnesini* kopyalar; `sayfaSayisi_` ve türetilmiş davranış **kesilir**, `m.yazdir()` taban sürümünü basar. Çözüm: **işaretçi/referansla** çalışın.

» `virtual` ve Dinamik Gönderim

Statik Tip vs Dinamik Tür

Taban `yazdir() virtual` bildirilirse, `p->yazdir()` çağrısı *çalışma zamanında* nesnenin gerçek türüne göre çözülür (`BasiliKitap::yazdir`); değilse statik tipe (`Materyal`) bağlanır (kumandanın "aç" tuşu gibi: arayüz aynı, davranış cihaza göre). Türetilmişte `void yazdir() const override;` yazın; imza eşleşmezse derleyici **hata** verir — sessiz "yeni fonksiyon" hatasını yakalar.

» Sanal Yıkıcı NEDEN Gerekli

Dikkat

`Materyal* p = new BasiliKitap(...); delete p;` — `~Materyal() virtual` **DEĞİLSE** yalnız taban yıkıcısı çalışır; türetilmişin ek kaynağı sızar (tanımsız davranış). Taban işaretçiyle silinen her polimorfik tabanın yıkıcısı `virtual` olmalıdır.

» Saf Sanal (`= 0`) ve Soyut Sınıf

```
class Sekil { // SOYUT
public:
  virtual double alan() const = 0; // saf sanal
  virtual ~Sekil() = default;
};
class Daire : public Sekil {
public:
  double alan() const override; // ZORUNLU
};
```

Sözleşme Dayatmak

`= 0` tabanın işlevi *tanımlamadığını*, her türetilmişin tanımlamak *zorunda* olduğunu söyler. En az bir saf sanalı olan sınıf **soyut**tur — doğrudan örneklenemez (`Sekil s;` derleme hatası), yalnız türetilmişleri örneklenir.

» Çokbiçimli Koleksiyon

```
class Materyal { // SOMUT taban
public:
  virtual void yazdir() const; // canlanan cagri
  virtual ~Materyal(); // sanal YIKICI
protected:
  Yazar yazar_; // has-a: Yazar'a sahip
};
class Kutuphane { // has-a koleksiyon
  std::vector<std::unique_ptr<Materyal>> raf_;
public:
  void ekle(std::unique_ptr<Materyal> m);
  void hepsiniYazdir() const {
    for (const auto& m : raf_)
      m->yazdir(); // her eleman GERCEK turune gore
  }
};
```

» Kalıtım (is-a) vs Kompozisyon (has-a)

Test	İlişki	Örnek
"...dır/...tir"	is-a	`BasiliKitap` bir `Materyal`
"...sahiptir"	has-a	`Kutuphane` raflara sahip
"...içerir"	has-a	`Materyal`'in bir `Yazar`'ı

Seçim Ölçütü

"...dır" doğalsa **kalıtım**; "...sahiptir/...içerir" doğalsa **kompozisyon**. `Kutuphane` bir `std::vector` *değildir*, bir vektöre *sahiptir* — `vector`'dan türetmeyin. **Belirsizse kompozisyonu tercih edin.**

» Sık Yapılan Hatalar

Ortak Hata / Tuzak

- Polimorfik tabanda yıkıcıyı `virtual` yapmamak → silmede kaynak sızar.
- Türetilmişi *değerle* taban değişkene atamak → slicing.
- `override` yazmayıp imzayı yanlış yazmak → sessizce yeni fonksiyon oluşur.
- `std::vector<Materyal>` çokbiçimli tutmak → dilimler; `unique_ptr` kullanın.

5 • İzleme **Nerede Kullanılır:** Ortak veri/davranışı olan *ilişkili* tipleri tek arayüz altında toplayıp bir kapta çokbiçimli işlemek istediğinizde: is-a için `public` kalıtım + `virtual`/`override` + sanal yıkıcı; "her türetilmiş uygulasın" sözleşmesi için saf sanal (`= 0`); farklı türleri tek döngüde davrandırmak için `std::vector<std::unique_ptr<Taban>>`. Bir tip başkasını yalnız *içeriyorsa* (has-a) kompozisyon (üye) kullanın.